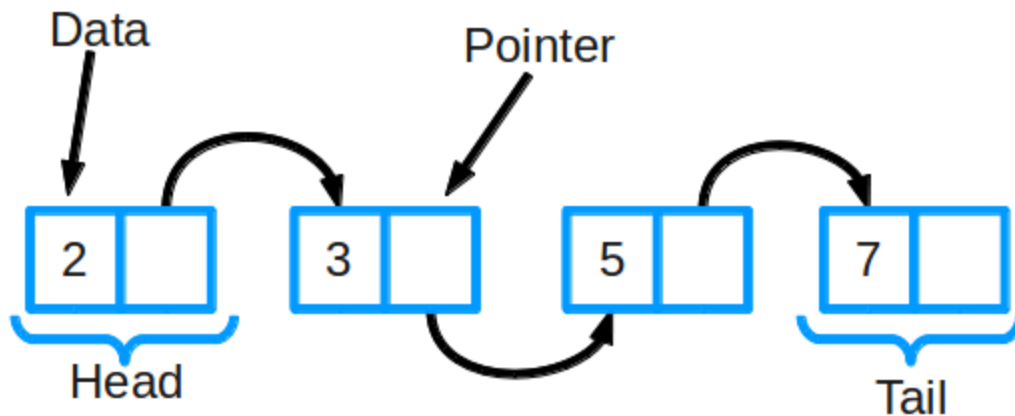


Data Structure

Linked List



A linked list consists of a group of nodes which together represent a sequence. Each node contains two things: the actual data being stored (which can be basically any type of data) and a pointer (or link) to the next node in the sequence. There are also doubly linked lists where each node has a pointer to both the next item and the previous item in the list.

The most basic operations in a linked list are adding an item to the list, deleting an item from the list, and searching the list for an item.

freeCodeCamp challenges

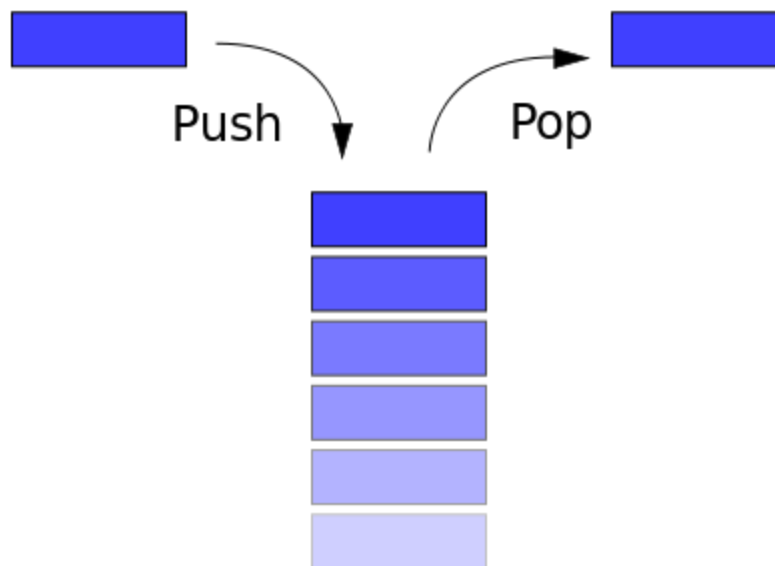
- [Work with Nodes in a Linked List](#)
- [Create a Linked List Class](#)
- [Remove Elements from a Linked List](#)
- [Search within a Linked List](#)
- [Remove Elements from a Linked List by Index](#)
- [Add Elements at a Specific Index in a Linked List](#)
- [Create a Doubly Linked List](#)

- [Reverse a Doubly Linked List](#)

Stack

A stack is a basic data structure where you can only insert or delete items at the top of the stack.

The stack is considered LIFO (Last In First Out) — meaning the last item you put in the stack is the first item that comes out of the stack



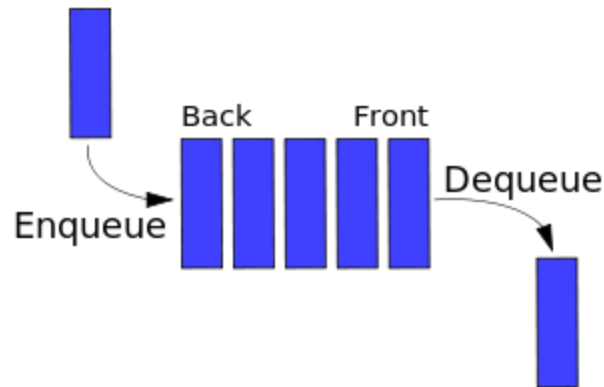
There are three main operations that can be performed on stacks: inserting an item into a stack (called 'push'), deleting an item from the stack (called 'pop'), and displaying the contents of the stack (sometimes called 'pip').

freeCodeCamp challenges

- [Learn how a Stack Works](#)
- [Create a Stack Class](#)

Queues

You can think of a queue as a line of people at a grocery store. The first one in the line is the first one to be served. Just like a queue.



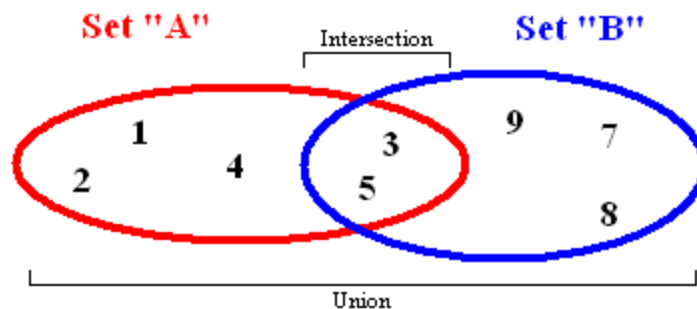
A queue is considered FIFO (First In First Out) to demonstrate the way it accesses data. This means that once a new element is added, all elements that were added before have to be removed before the new element can be removed.

A queue has just two main operations: enqueue and dequeue. Enqueue means to insert an item into the back of the queue and dequeue means removing the front item.

freeCodeCamp challenges

- [Create a Queue Class](#)
- [Create a Priority Queue Class](#)
- [Create a Circular Queue](#)

Sets



The set data structure stores values without any particular order and with no repeated values.

- Union — This combines all the items from two different sets and returns this as a new set (with no duplicates).
- Intersection — Given two sets, this function returns another set that has all items that are part of both sets.
- Difference — This returns a list of items that are in one set but NOT in a different set.
- Subset — This returns a boolean value that shows if all the elements in one set are included in a different set.

freeCodeCamp challenges

- [Create a Set Class](#)
- [Remove from a Set](#)
- [Size of the Set](#)
- [Perform a Union on Two Sets](#)
- [Perform an Intersection on Two Sets of Data](#)
- [Perform a Difference on Two Sets of Data](#)
- [Perform a Subset Check on Two Sets of Data](#)
- [Create and Add to Sets in ES6](#)
- [Remove items from a set in ES6](#)
- [Use .has and .size on an ES6 Set](#)
- [Use Spread and Notes for ES5 Set\(\) Integration](#)

Maps

A map is a data structure that stores data in key / value pairs where every key is unique. A map is sometimes called an associative array or dictionary. It is often used for fast look-ups of data. Maps allow the following things:

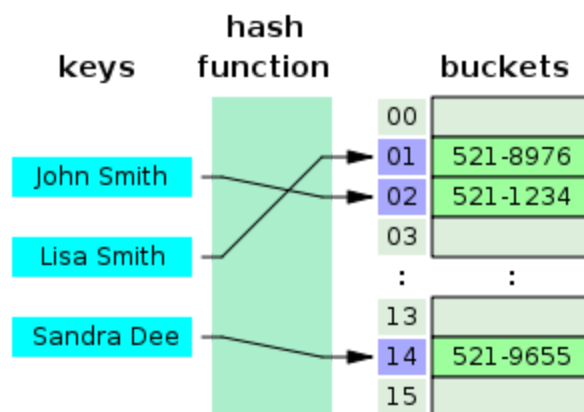
	KEYS	VALUES	
	Jan	327.2	
	Feb	368.2	
	Mar	197.6	
	Apr	178.4	
	May	100.0	
	Jun	69.9	
	Jul	32.3	
Aug →	Aug	37.3	→ 37.3
	Sep	19.0	
	Oct	37.0	
	Nov	73.2	
	Dec	110.9	
	Annual	1551.0	

- the addition of a pair to the collection
- the removal of a pair from the collection
- the modification of an existing pair
- the lookup of a value associated with a particular key

freeCodeCamp challenges

- [Create a Map Data Structure](#)
- [Create an ES6 JavaScript Map](#)

Hash Tables



A hash table is a map data structure that contains key / value pairs.

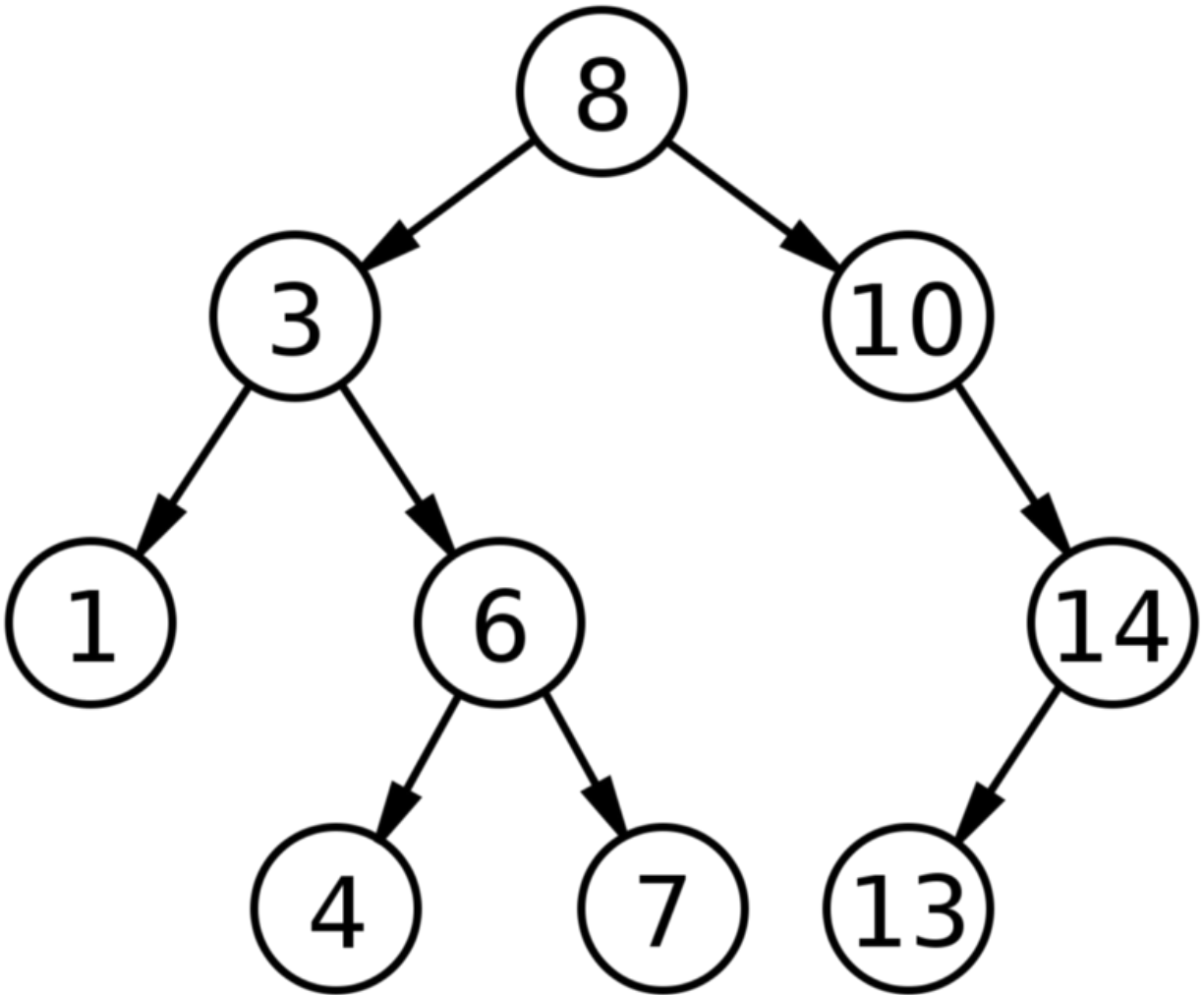
The hash function usually takes a string as input and it outputs an numerical value. The hash function should always give the same output number for the same input. When two inputs hash to the same numerical output, this is called a collision. The goal is to have few collisions.

So when you input a key / value pair into a hash table, the key is run through the hash function and turned into a number. This numerical value is then used as the actual key that the value is stored by. When you try to access the same key again, the hashing function will process the key and return the same numerical result. The number will then be used to look up the associated value. This provides very efficient $O(1)$ lookup time on average.

freeCodeCamp challenges

- [Create a Hash Table](#)

Binary Search Tree



A tree is a data structure composed of nodes. It has the following characteristics:

1. Each tree has a root node (at the top).
2. The root node has zero or more child nodes.
3. Each child node has zero or more child nodes, and so on.

A *binary search* tree adds these two characteristics:

1. Each node has up to two children.
2. For each node, its left descendants are less than the current node, which is less than the right descendants.

Binary search trees allow fast lookup, addition and removal of items. The way that they are set up means that, on average, each comparison allows the operations to

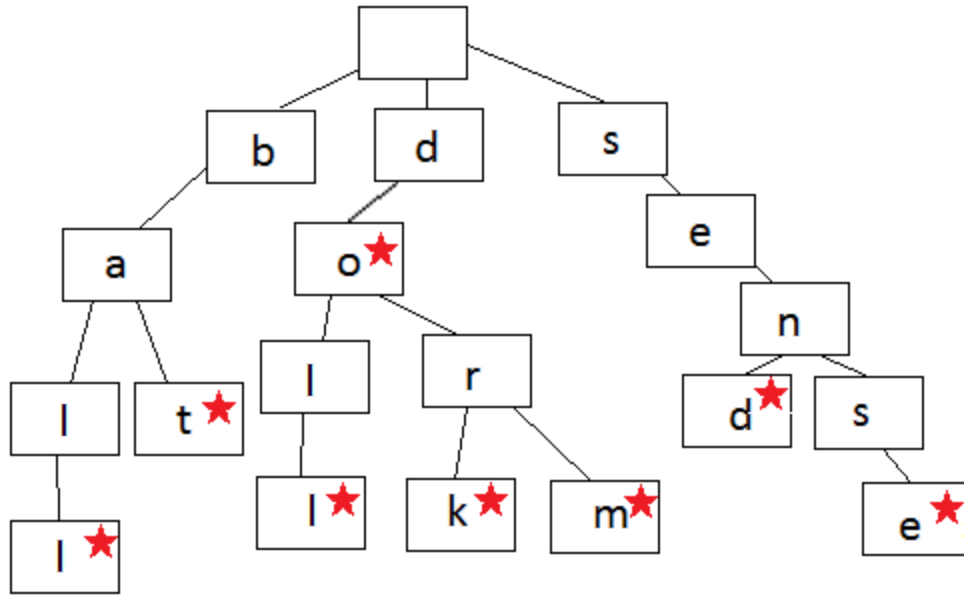
skip about half of the tree, so that each lookup, insertion or deletion takes time proportional to the logarithm of the number of items stored in the tree.

freeCodeCamp challenges

- [Find the Minimum and Maximum Value in a Binary Search Tree](#)
- [Add a New Element to a Binary Search Tree](#)
- [Check if an Element is Present in a Binary Search Tree](#)
- [Find the Minimum and Maximum Height of a Binary Search Tree](#)
- [Use Depth First Search in a Binary Search Tree](#)
- [Use Breadth First Search in a Binary Search Tree](#)
- [Delete a Leaf Node in a Binary Search Tree](#)
- [Delete a Node with One Child in a Binary Search Tree](#)
- [Delete a Node with Two Children in a Binary Search Tree](#)
- [Invert a Binary Tree](#)

Trie

The trie (pronounced 'try'), or prefix tree, is a kind of search tree. A trie stores data in steps where each step is a node in the trie. Tries are often used to store words for quick lookup, such as a word auto-complete feature.



Each node in a language trie contains one letter of a word. You follow the branches of a trie to spell a word, one letter at a time. The steps begin to branch off when the order of the letters diverge from the other words in the trie, or when a word ends. Each node contains a letter (data) and a boolean that indicates whether the node is the last node in a word.

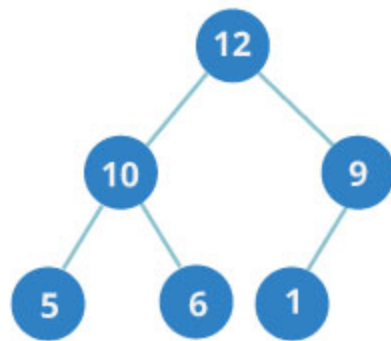
Look at the image and you can form words. Always start at the root node at the top and work down. The trie shown here contains the word ball, bat, doll, do, dork, dorm, send, sense.

freeCodeCamp challenges

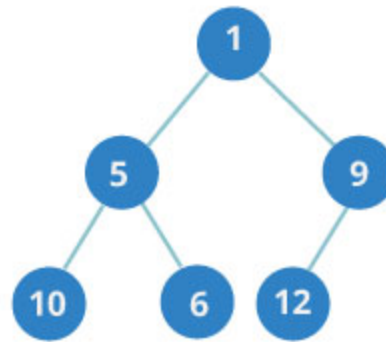
- [Create a Trie Search Tree](#)

Binary Heap

A binary heap is another type of tree data structure. Every node has at most two children. Also, it is a complete tree.



Max Heap



Min Heap

A binary heap can be either a min heap or a max heap. In a max heap, the keys of parent nodes are always greater than or equal to those of the children. In a min heap, the keys of parent nodes are less than or equal to those of the children.

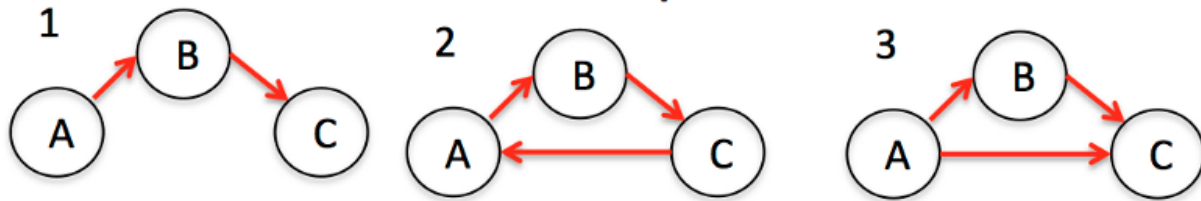
The order between levels is important but the order of nodes on the same level is not important. In the image, you can see that the third level of the min heap has values 10, 6, and 12. Those numbers are not in order.

Graph

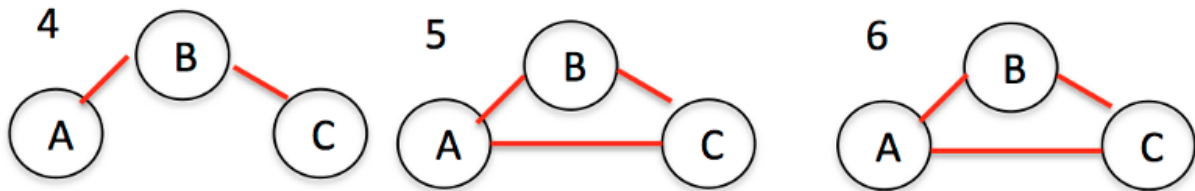
Graphs are collections of nodes (also called vertices) and the connections (called edges) between them. Graphs are also known as networks.

One example of graphs is a social network. The nodes are people and the edges are friendship.

Directed Graphs

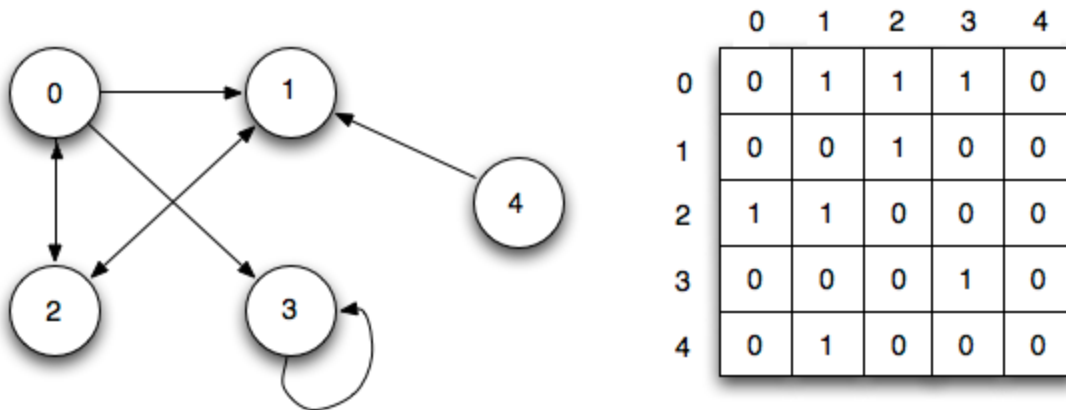


Undirected Graphs



There are two major types of graphs: directed and undirected. Undirected graphs are graphs without any direction on the edges between nodes. Directed graphs, in contrast, are graphs with a direction in its edges.

Two common ways to represent a graph are an adjacency list and an adjacency matrix.



An adjacency list can be represented as a list where the left side is the node and the right side lists all the other nodes it's connected to.

An adjacency matrix is a grid of numbers, where each row or column represents a different node in the graph.

Zeros mean there is no edge or relationship. Ones mean there is a relationship. Numbers higher than one can be used to show different weights.

Traversal algorithms are algorithms to traverse or visit nodes in a graph. The main types of traversal algorithms are breadth-first search and depth-first search. One of the uses is to determine how close nodes are to a root node.

freeCodeCamp challenges

- [Adjacency List](#)
- [Adjacency Matrix](#)
- [Incidence Matrix](#)
- [Breadth-First Search](#)
- [Depth-First Search](#)

Grokking Algorithms by Aditya Y. Bhargava - PDF Drive

publisher's website at www.manning.com/books/grokking-algorithms Purchase of Grokking Algorithms includes free access to a private web.

 <https://www.pdfdrive.com/grokking-algorithms-e105413330.html>

